

Concurrency Essentials

: Callable & Future; CompletableFuture basics; when to use async

Rahul Guha || SME || Java FS Training || HCL
2. Modern Java (8–21)

Concurrency Essentials in Java

Callable & Future • CompletableFuture • When to Use Async

Concurrency allows a Java program to perform multiple tasks concurrently, improving responsiveness and resource utilization. Java provides several APIs for handling concurrent tasks, especially through the `java.util.concurrent` package.

Callable Interface `Callable<V>` is similar to `Runnable`, but it has two important advantages:

1. It returns a result.
2. It can throw checked exceptions.

Syntax:

```
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception;
}
```

Feature	Runnable	Callable
Package	<code>java.lang</code>	<code>java.util.concurrent</code>
Method	<code>run()</code>	<code>call()</code>
Return value	No	Yes
Checked exception	Cannot throw directly	Can throw
Used with	Thread, ExecutorService	ExecutorService
Generic type	No	Yes

Concurrency Essentials in Java

Callable Returning a Result

```
import java.util.concurrent.*;

public class CallableExample {

    public static void main(String[] args) throws
Exception {

        ExecutorService service =
Executors.newSingleThreadExecutor();

        Callable<Integer> task = () -> {
            System.out.println("Calculating...");
            Thread.sleep(2000);
            return 100 + 200;
        };
    }
}
```

```
Future<Integer> future = service.submit(task);

System.out.println("Task submitted...");

Integer result = future.get();

System.out.println("Result = " + result);

service.shutdown();
}
```

Task submitted...
Calculating...
Result = 300

Concurrency Essentials in Java

2. Future Interface

Future<V> represents the result of an asynchronous computation.

When we submit a Callable to an ExecutorService, we receive a Future.

```
Future<Integer> future =  
service.submit(task);
```

Task



ExecutorService



Future



Result

Method	Purpose
get()	Gets the result; waits if necessary
get(timeout, unit)	Waits only for specified time
isDone()	Checks whether task is completed
isCancelled()	Checks whether task was cancelled
cancel()	Attempts to cancel task

Concurrency Essentials in Java

3. Future get()

Integer result = future.get();

If the task has not finished, the current thread **blocks** until the result becomes available.

.....

```
Future<Integer> future =
```

```
service.submit(() -> {
```

```
    Thread.sleep(3000);
```

```
    return 500;
```

```
});
```

```
System.out.println("Waiting...");
```

```
System.out.println(future.get());
```

The program waits approximately 3 seconds.

4. Future with Timeout

Instead of waiting indefinitely: Integer result = future.get(2, TimeUnit.SECONDS);

```
import java.util.concurrent.*;
```

```
public class FutureTimeout {
```

```
    public static void main(String[] args) {
```

```
        ExecutorService service = Executors.newSingleThreadExecutor();
```

```
        Future<Integer> future = service.submit(() -> {
```

```
            Thread.sleep(5000);
```

```
            return 100;
```

```
        });
```

Concurrency Essentials in Java

```
try {  
  
    Integer result = future.get(2, TimeUnit.SECONDS);  
    System.out.println(result);  
  
} catch (TimeoutException e) {  
  
    System.out.println("Task took too long.");  
} catch (Exception e) {  
    e.printStackTrace();  
} finally {  
  
    service.shutdown();  
}  
}  
}
```

Concurrency Essentials in Java

5. Checking Task Status

```
System.out.println(future.isDone());
```

Example():

```
Future<Integer> future = service.submit(() -> {  
    Thread.sleep(2000);  
    return 100;  
});
```

```
System.out.println("Done? " + future.isDone());
```

```
Thread.sleep(3000);
```

```
System.out.println("Done? " + future.isDone());
```

Output:

Done?

false

true

6. Cancelling a Future

```
future.cancel(true);
```

```
import java.util.concurrent.*;
```

```
public class FutureCancel {  
    public static void main(String[] args) {  
        ExecutorService service =  
            Executors.newSingleThreadExecutor();
```

```
        Future<Integer> future = service.submit(() -> {  
            for (int i = 1; i <= 10; i++) {  
                System.out.println(i);  
                Thread.sleep(1000);  
            }  
            return 100;  
        });
```

Concurrency Essentials in Java

```
try {  
    Thread.sleep(2500);  
  
    future.cancel(true);  
    System.out.println("Task cancelled.");  
} catch (Exception e) {  
    e.printStackTrace();  
} finally {  
  
    service.shutdown();  
}  
}
```

cancel(true) attempts to interrupt a running task.

7. Multiple Callable Tasks

One major advantage of **ExecutorService** is that **multiple tasks can execute concurrently**.

```
import java.util.*;  
import java.util.concurrent.*;  
public class MultipleCallable {  
    public static void main(String[] args) throws Exception {  
        ExecutorService service =  
            Executors.newFixedThreadPool(3);  
        Callable<Integer> task1 = () -> 10 + 20;  
        Callable<Integer> task2 = () -> 30 + 40;  
        Callable<Integer> task3 = () -> 50 + 60;  
        Future<Integer> f1 = service.submit(task1);  
        Future<Integer> f2 = service.submit(task2);  
        Future<Integer> f3 = service.submit(task3);  
        System.out.println(f1.get()); System.out.println(f2.get());  
        System.out.println(f3.get()); service.shutdown();  
    }  
}
```

Concurrency Essentials in Java

8. `invokeAll()` If you have multiple Callable tasks and want to execute all of them:

```
List<Future<T>> futures =
```

```
executor.invokeAll(tasks);
```

```
import java.util.*;
```

```
import java.util.concurrent.*;
```

```
public class InvokeAllExample {
```

```
    public static void main(String[] args) throws  
    Exception {
```

```
        ExecutorService service =  
        Executors.newFixedThreadPool(3);
```

```
List<Callable<Integer>> tasks = Arrays.asList(  
    () -> 10 * 10,  
    () -> 20 * 20,  
    () -> 30 * 30  
);
```

```
List<Future<Integer>> results =
```

```
    service.invokeAll(tasks);
```

```
    for (Future<Integer> future : results) {
```

```
        System.out.println(future.get());
```

```
    }
```

```
    service.shutdown();
```

```
    }
```

```
}
```

CPU-Bound vs I/O-Bound Tasks

Task Type	Examples	Async/Concurrency
CPU-bound	Image processing	Useful with controlled parallelism
CPU-bound	Mathematical computation	Use appropriate thread pool
I/O-bound	API calls	Highly suitable
I/O-bound	Database calls	Highly suitable
I/O-bound	File/network operations	Often suitable
Waiting	External service response	Suitable

Callable vs Future vs CompletableFuture

Feature	Callable	Future	CompletableFuture
What is it?	Task	Handle to result	Async computation stage
Returns result	☑	Represents result	☑
Executes task	Defines task	No	Can initiate async task
Chaining	✗	✗	☑
Combination	✗	Limited	☑
Exception handling	Via exception	get() exceptions	exceptionally(), handle()
Non-blocking style	✗	Limited	☑
Main use	Define computation	Retrieve/cancel result	Build async workflows